

Generative AI Chat Log

COMP30830 Dublin Bikes Web Application

Student: Bethireddy Glory Sruthi | ID: 25203412

This document records ten prompts submitted to a generative AI assistant during the development of the Dublin Bikes Web Application for COMP30830. Each entry documents the development challenge that motivated the query, the AI guidance received, the code or approach suggested, and how that guidance was applied — or adapted — in the final project. The log demonstrates how AI tools were used as a learning and debugging resource rather than a replacement for independent engineering judgement.

1. Pandas to MariaDB Bulk Insert

Prompt: How do I efficiently insert a large CSV into MariaDB using Python without slowing down the system?

Context and Motivation

The project required ingesting 298,946 rows of historical bike and weather data into the MariaDB database. A naive row-by-row insert would have taken hours and locked the database during ingestion, making it unusable for concurrent Flask requests.

AI Guidance

The AI recommended chunked insertion using `pandas.read_csv()` with a `chunksize` parameter combined with `cursor.executemany()`. This approach reads the CSV progressively in memory-safe batches of 5,000 rows rather than loading the entire file at once, then sends each batch to MariaDB in a single round trip rather than one query per row.

```
import pandas as pd
import mysql.connector

conn = mysql.connector.connect(...)
cursor = conn.cursor()

chunks = pd.read_csv("data.csv", chunksize=5000)

for chunk in chunks:
    data = list(chunk.itertuples(index=False, name=None))
    cursor.executemany(
        "INSERT INTO stations VALUES (%s, %s, %s, %s)",
        data
    )
conn.commit()
```

Three additional optimisations were highlighted. First, batch size should be tuned between 500 and 5,000 rows — smaller batches reduce memory pressure but increase round trips, while larger batches do the opposite. Second, `autocommit` should be disabled during bulk load so the database engine can defer disk flushes until each batch is committed explicitly, dramatically reducing I/O overhead. Third,

indexes should only be added after ingestion completes — maintaining an index during bulk insert forces the engine to update the B-tree structure on every row, which can increase total ingestion time by an order of magnitude.

Application to Project

This pattern was applied directly when populating the `station_snapshot` and `weather_log` tables from the December 2024 historical dataset, reducing ingestion time from an estimated several hours to under ten minutes.

2. SQL Query Optimisation

Prompt: My query joining weather and station tables is slow. How do I optimise it?

Context and Motivation

The Flask backend frequently executes queries that join `station_status` and `weather` on a shared timestamp column to assemble the feature vectors sent to the ML model. Without optimisation, full table scans on both tables were causing latency spikes visible to users in the frontend chart rendering.

AI Guidance

The AI identified the root cause as missing indexes on the join columns and recommended creating composite indexes on both tables:

```
CREATE INDEX idx_station_time ON station_status(station_id, timestamp);
CREATE INDEX idx_weather_time ON weather(timestamp);

-- Optimised query
SELECT s.station_id, w.temperature, s.bikes_available
FROM station_status s
JOIN weather w ON s.timestamp = w.timestamp;
```

The composite index on `station_status` is ordered as `(station_id, timestamp)` rather than `(timestamp, station_id)` deliberately — since queries almost always filter by a specific station before scanning time, the leftmost column of the index should be the one with highest selectivity for that access pattern. The AI also recommended rewriting the query to use explicit column selection rather than `SELECT *`, avoiding unnecessary data transfer across the query boundary.

A critical warning was also flagged: join columns must share the same data type. A mismatch — for example, timestamp stored as `VARCHAR` in one table and `DATETIME` in the other — forces an implicit cast on every row comparison, which bypasses the index entirely and reverts to a full scan.

Application to Project

After applying these indexes, the average query execution time for the prediction endpoint dropped significantly, making the `/predict` API response feel instantaneous to end users.

3. Feature Engineering Code

Prompt: How do I generate time-based features in Python for ML?

Context and Motivation

Raw timestamps are not meaningful to a machine learning model — an integer like 1733050800 carries no signal about rush hour or weekday commuting patterns. The temporal features needed to be extracted and, where appropriate, transformed before being passed to the model.

AI Guidance

The AI demonstrated using pandas datetime accessors to extract hour and day_of_week from a parsed timestamp column, and introduced the concept of cyclical encoding:

```
import pandas as pd
import numpy as np

df['timestamp'] = pd.to_datetime(df['timestamp'])
df['hour'] = df['timestamp'].dt.hour
df['day_of_week'] = df['timestamp'].dt.dayofweek

# Cyclical encoding — avoids discontinuity at midnight
df['hour_sin'] = np.sin(2 * np.pi * df['hour'] / 24)
df['hour_cos'] = np.cos(2 * np.pi * df['hour'] / 24)
```

The most important concept introduced here was cyclical encoding. A raw integer hour ranging from 0 to 23 implies to the model that hour 0 (midnight) and hour 23 (11pm) are maximally distant — when in reality they are adjacent. Encoding hour as a sine/cosine pair on a unit circle preserves this circular continuity, so the model correctly learns that late-night and early-morning hours share similar low-usage characteristics.

Application to Project

While cyclical encoding was explored during experimentation, the final deployed model uses raw integer features for hour, day_of_week, is_weekend, and month. The Decision Tree's splitting mechanism is not distance-sensitive, so it does not suffer from the discontinuity problem that motivates cyclical encoding for distance-based models like k-NN or linear regression. The AI guidance was nonetheless valuable for ruling out an approach that would have added complexity without benefit for this model family.

4. Train/Test Split with Time Series Constraint

Prompt: How do I split a dataset for time-series ML without data leakage?

Context and Motivation

A naive random train/test split — the default in scikit-learn's `train_test_split` — would have allowed future data points to appear in the training set, artificially inflating evaluation metrics and producing a model that appears to perform better than it actually would in production.

AI Guidance

The AI explained that time-series datasets must always be split chronologically, not randomly. The correct approach sorts the dataframe by timestamp first and then slices at a fixed point:

```
df = df.sort_values('timestamp')

train_size = int(len(df) * 0.8)
```

```
train = df[:train_size]
test = df[train_size:]

# Key rule: future data must NEVER appear in the training set
```

This ensures the model is always evaluated on data it has never seen during training and that the test set represents a genuine future relative to the training window — which mirrors real production conditions far more faithfully than random splitting.

Application to Project

This chronological split was applied to the December 2024 dataset, meaning the model was trained on approximately the first 24 days of December and evaluated on the final six days. The resulting MAE and R^2 figures are therefore conservative, honest estimates of production performance rather than optimistic in-sample metrics.

5. DecisionTreeRegressor Training

Prompt: How do I train a DecisionTreeRegressor properly for this dataset?

Context and Motivation

After benchmarking multiple algorithms, the Decision Tree was identified as the best candidate for deployment given the EC2 memory constraints. However, guidance was needed on how to configure hyperparameters to avoid overfitting while preserving predictive accuracy.

AI Guidance

The AI demonstrated a clean training pipeline with hyperparameter controls:

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split

X = df[features]
y = df['bikes_available']

model = DecisionTreeRegressor(
    max_depth=8,
    min_samples_split=10,
    random_state=42
)

model.fit(X_train, y_train)
```

Key parameters explained: `max_depth` limits how many levels of splits the tree can make, preventing it from memorising training data. `min_samples_split` requires at least 10 samples before a node can be split further, which prevents the tree from creating leaf nodes that represent only one or two data points. `random_state=42` ensures reproducibility across training runs.

Application to Project

During experimentation, `max_depth=25` was tested first (MAE=1.636, $R^2=0.881$) followed by removing the depth cap entirely, which improved the model to MAE=1.013 and $R^2=0.929$. The unlimited depth was acceptable for this problem because the dataset is large (298,946 rows) and the

tree needed to learn highly specific per-station, per-hour rules that shallow splits cannot represent. The final model's 13 MB size confirmed the tree had grown large but remained deployable.

6. Model Serialisation and Reload Bug Fix

Prompt: My saved model gives different predictions after reload. Why?

Context and Motivation

During development, the model was saved with pickle and reloaded during Flask startup, but predictions differed from those produced during training evaluation. This is a subtle and dangerous bug — the model appeared to work, but was silently returning wrong values.

AI Guidance

The AI identified feature order mismatch as the most common cause. When a pandas DataFrame is constructed at inference time, column order is not guaranteed to match the order used during training unless enforced explicitly. The fix is to save the exact feature list alongside the model and always reconstruct the input DataFrame using that saved list:

```
import joblib
import json

# Save
joblib.dump(model, "model.pkl")

# Load and use safely
model = joblib.load("model.pkl")
features = json.load(open("model_features.json"))
X = df[features] # enforces correct column order
```

joblib is preferred over pickle for scikit-learn models because it uses memory-mapped I/O for large numpy arrays, making serialisation and loading substantially faster for tree-based models.

Application to Project

A `model_features.json` file was added to the repository storing the exact eight-feature list in training order. Flask reads this file at startup before loading the model, and every call to `/predict` reconstructs the input DataFrame using `df[features]` to guarantee alignment. This eliminated the prediction drift entirely.

7. Flask Prediction Endpoint

Prompt: How do I write a clean Flask endpoint for ML prediction?

Context and Motivation

The prediction model needed to be exposed as a REST API so the JavaScript frontend could call it from the browser, pass station and time parameters, and receive a predicted bike count to display on the map.

AI Guidance

The AI provided a minimal but production-ready endpoint structure:

```

from flask import Flask, request, jsonify
import joblib
import numpy as np

app = Flask(__name__)
model = joblib.load("model.pkl") # loaded ONCE at startup

@app.route('/predict', methods=['POST'])
def predict():
    data = request.json

    features = np.array([
        data['hour'],
        data['temperature'],
        data['humidity']
    ]).reshape(1, -1)

    prediction = model.predict(features)

    return jsonify({"predicted_bikes": float(prediction[0])})

```

Critical details: the model is loaded once at application startup rather than on every request — loading a 13 MB pickle file on each API call would add hundreds of milliseconds of latency. `reshape(1, -1)` converts the flat feature array into the 2D shape scikit-learn's `predict()` expects. The prediction is cast to float before JSON serialisation because numpy scalar types are not natively JSON-serialisable.

Application to Project

The actual deployed endpoint extends this pattern by also calling the OpenWeather API to fetch live weather data, extracting temporal features from the requested datetime, and building the input DataFrame using `model_features.json` before calling `model.predict()`.

8. API Input Validation Bug Fix

Prompt: My Flask API crashes when input is missing. How do I fix it?

Context and Motivation

During integration testing, the `/predict` endpoint raised unhandled `KeyError` exceptions when the frontend sent requests with missing parameters, returning a 500 Internal Server Error to the client with no useful diagnostic information.

AI Guidance

The AI recommended adding an explicit validation layer at the top of the endpoint handler that checks for all required fields before attempting inference:

```

required_fields = ['hour', 'temperature', 'humidity']

for field in required_fields:
    if field not in data:
        return jsonify({"error": f"{field} missing"}), 400

```

This returns a structured 400 Bad Request response with a clear error message rather than a raw stack trace, making it far easier for the frontend JavaScript to handle failures gracefully and display a user-friendly error rather than a broken chart.

Application to Project

Input validation was extended in the final implementation to also check data types — for example, rejecting non-numeric values for hour or temperature — and to validate range bounds (e.g. hour must be between 0 and 23). This hardened the API against both accidental frontend bugs and malformed requests.

9. pytest for API Testing

Prompt: How do I write pytest cases for the Flask prediction API?

Context and Motivation

Manual testing of the /predict endpoint was not reproducible and could not be run automatically as part of the development workflow. Automated tests were needed to catch regressions when the model or endpoint logic changed.

AI Guidance

The AI demonstrated a simple integration test using the requests library that sends a real HTTP POST to the running Flask server and asserts on the response:

```
import requests

def test_prediction():
    response = requests.post("http://localhost:5000/predict", json={
        "hour": 10,
        "temperature": 22,
        "humidity": 60
    })

    assert response.status_code == 200
    assert "predicted_bikes" in response.json()

# Edge cases to cover:
# - Missing fields → expect 400
# - Invalid types → expect 400
# - Extreme values → expect non-crashing response
```

The AI also highlighted the edge cases that need dedicated test functions: missing fields (should return 400), invalid data types such as a string where a float is expected (should return 400), and extreme values such as hour=99 or temperature=-999 (should either return a validation error or a clamped, non-crashing prediction).

Application to Project

These test patterns were adopted in the project's test suite, covering the happy path, missing field handling, and type validation, giving confidence that endpoint changes do not silently break prediction behaviour.

10. EC2 Deployment Debugging

Prompt: My Flask app runs locally but not on EC2. What should I check?

Context and Motivation

After deploying to the AWS EC2 instance, the Flask application was unreachable from the browser despite appearing to start successfully in the SSH session. This blocked the entire project from going live.

AI Guidance

The AI provided a structured four-point checklist covering the most common causes of this exact failure mode.

1. Port binding — Flask defaults to `host="127.0.0.1"`, which only accepts connections from localhost. On EC2 the app must bind to `0.0.0.0` to accept external traffic:

```
app.run(host="0.0.0.0", port=5000)
```

2. Security group — AWS EC2 instances block all inbound traffic by default. An inbound rule must be added in the EC2 console explicitly allowing TCP traffic on port 5000 (or port 80 if running behind a reverse proxy).

3. Firewall — Ubuntu's `ufw` firewall may be active independently of the AWS security group, blocking the port at the OS level:

```
sudo ufw allow 5000
```

4. Log inspection — if the app still fails to start, `journalctl` surfaces the actual error from the `systemd` service log:

```
journalctl -u flask.service
```

Application to Project

The root cause in this project was the combination of the first two issues — Flask binding to localhost and the EC2 security group not having port 5000 open. The AI checklist guided the fix in under ten minutes. The `journalctl` pattern was also useful later when debugging the model memory crash caused by the Random Forest's 933 MB serialised size, which surfaced as an OOM kill in the service log.

End of AI chat log. All ten entries represent real development queries made during the project. In each case, the AI response was evaluated critically before adoption — several suggestions (e.g. cyclical encoding, max_depth capping) were tested and ultimately rejected or adapted based on the specific constraints of the Dublin Bikes system.